



Integrating Chatbot app to Android image

Submitted to: Dr Abdelrhman Abotaleb & Dr Ahmed Hussein

Team: Android

Done by:

Mohamed Ahmed Abdelazim Mohamed

&

Mohamed Ismail Sebea

Overview

First, we used python (instead Qt) for the GUI. [5]

- We built application using: Kivy for the graphical user interface (GUI).
- The application allows the user to type or speak to the chatbot, choose between different AI models, and see/chat with responses interactively on a beautiful UI.

Project Structure

ANDROID_APP

app

main.py #Main file combining Frontend (Kivy App) + Backend starter
microphone.png
response.mp3

backend

chat.py #Full chatbot backend logic (LSTM, BERT models, etc.).
intent_classifier/
chatbot_model_LSTM.h5
label_encoder_LSTM.pkl
response.mp3
vectorizer_LSTM.pkl
intents.json

Project Flow

Case 1: Text Input Flow

1. Launch app → backend server starts → socket connection is made.
2. User types a text message into the input field → presses Enter.
3. Frontend sends the typed message to the backend through the socket.
4. Backend receives the text and processes it:
 - a) If the selected model is LLM → Predict intent using BERT model.
 - b) If the selected model is LSTM → Predict intent using LSTM model.
5. Backend selects a response:
 - a) Chooses a random response from the intent's predefined responses.
6. Backend sends the selected response back to the frontend.
7. Frontend displays the response in a chat bubble.
8. If Text-to-Speech (TTS) is enabled, the backend speaks the response aloud.

Case 2: Mic (Voice Input) Flow

1. User presses the Mic button → Frontend sends "start recording" to the backend.
2. Backend sets listening mode ON → Sends message: "I'm now listening. Please speak."
3. User speaks into the microphone. → Backend records and transcribes the user's voice → Converts speech to text using Google Speech Recognition API.
4. User releases the Mic button → Frontend sends "stop recording" command to the backend.
5. Backend sends the recognized text back to the frontend.
6. Frontend receives the recognized voice text → Automatically fills the text input field with the recognized text → Automatically sends the recognized text back to the backend (as if user typed it).
7. Backend processes the recognized text (same as text input flow)

Codes Explanation

1. main.py [10]

- acts as the frontend user interface and a backend launcher.
- It uses Kivy for building a graphical user interface and sockets to communicate with the local backend server.
- Features:
 - Kivy GUI with chat bubbles for user and bot messages.
 - Microphone input (push-to-talk).
 - Text-to-speech toggle (TTS).
 - Switching between two models (LLM or LSTM) dynamically during the conversation.
 - Starts backend server automatically when the app is launched.
- UI Layout:
 - Top Bar:
 - Model switcher spinner (LLM/LSTM).
 - "New Chat" button.
 - Center:
 - Chat area with scrollable chat bubbles.
 - Bottom Bar:
 - "Listen" button (toggle TTS).
 - Text input field.
 - Microphone button (record voice).

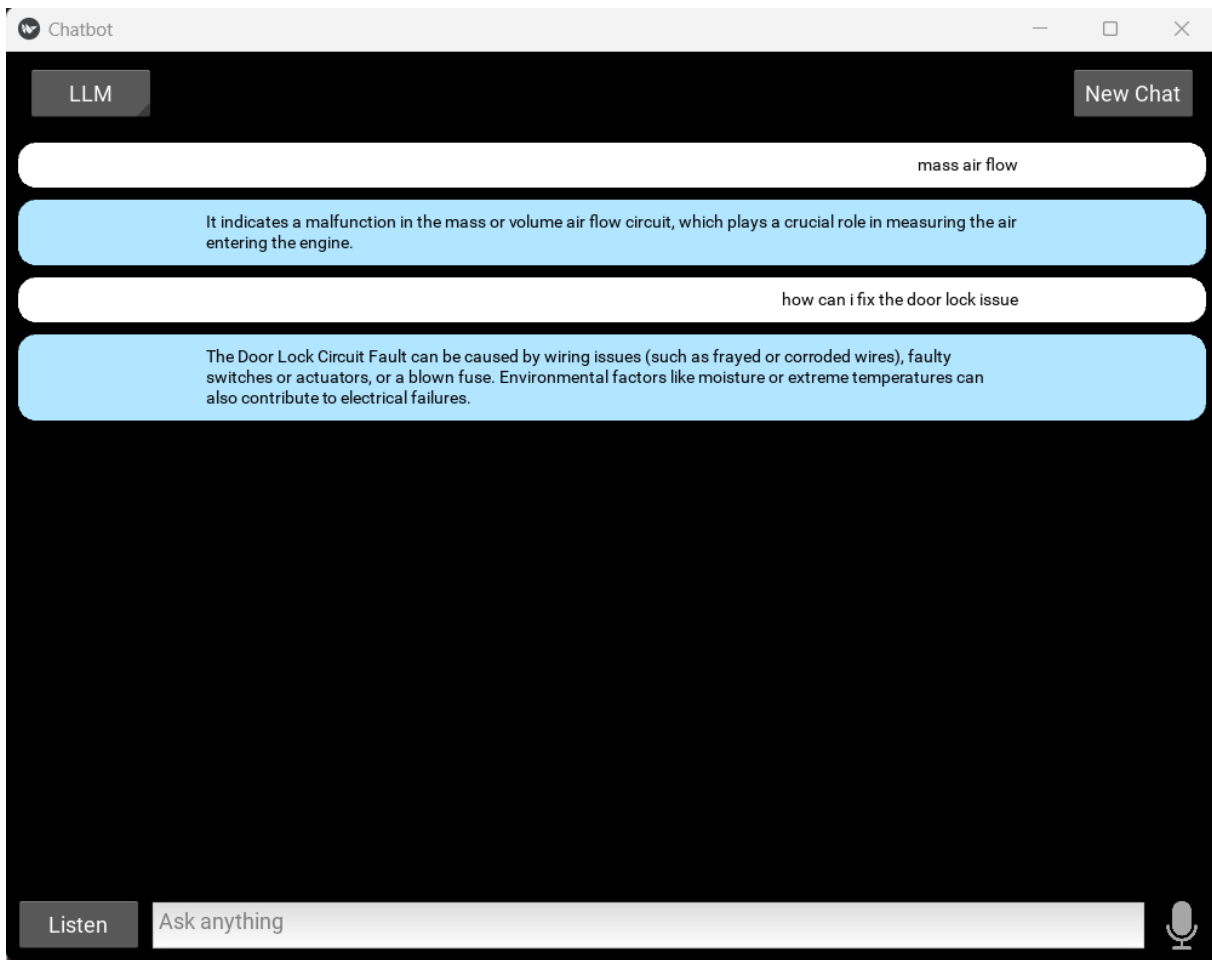


Figure 1 chatbot app

■ Socket Communication

- App connects to backend at 127.0.0.1:5800.
- Sends:
 - Text messages.
 - Voice commands (start recording, stop recording).
 - TTS toggle (toggle tts).
 - Model switch (llm or lstm).
- Receives:
 - Bot responses.
 - Voice recognition results (after recording).

2.chat.py [10]

- This file handles the natural language understanding of the chatbot.
- It accepts input from the frontend through a socket, predicts intent using LLM (BERT) or LSTM, and sends back the appropriate response.
- Features:
 - Predicts intent using BERT or LSTM based on user choice.
 - Provides random varied responses for each intent (to avoid repetition).
 - Supports TTS reading of bot responses.
 - Listens to voice input and transcribes it to text.
 - Allows switching between models at runtime.
- Functions:
 - `get_response()` : Retrieves and rotates responses for the detected intent.
 - `toggle_tts()` : Turns text-to-speech on or off.
 - `speak()` : Uses gTTS + pygame to say the bot's message.
 - `listen()` : Records user voice and converts it to text.
- Socket Server:
 - Listens on 127.0.0.1:5800
 - Accepts one client connection at a time.
 - Interprets commands:
 - Model change (llm, lstm).
 - Voice recording (start recording, stop recording).
 - TTS toggle (toggle tts).
 - Exit chat (exit, quit, bye).
 - Sends back bot responses or recognized voice text.
- Backend Workflow:
 1. Load models and prepare environment.
 2. Wait for connection from the frontend.

3. When message received:
 - a) If voice or control command → handle accordingly.
 - b) If text → predict intent → choose random response → send back.
4. Optionally speak the response (if TTS is enabled).

Converting the App to .apk file

1. Install Buildozer & some dependencies.[8]

```
sudo apt update

sudo apt install -y git zip unzip openjdk-17-jdk python3-pip autoconf libtool pkg-config
zlib1g-dev libncurses5-dev libncursesw5-dev libtinfo5 cmake libffi-dev libssl-dev

pip install --upgrade pip

pip install buildozer
```

2. Open project folder and initialize buildozer tool

```
cd android_app

buildozer init
```

3. Edit buildozer.spec. [7] [6] [9]

```
[app]

title = ChatbotApp

package.name = chatbotapp

package.domain = org.Abdelazim

source.dir = app/main.py

source.include_exts = py,png,mp3,json,pkl,h5,whl

requirements = python3,kivy,numpy,scikit-learn,keras,tensorflow-
cpu==2.9.1,gtts,speechrecognition,pygame==2.3.0
```

```
orientation = portrait
```

```
fullscreen = 1
```

```
android.archs = arm64-v8a
```

4. Build APK

```
buildozer -v android debug
```

5. Flash APK on raspi using adb command

```
adb install bin/<the bin name>
```

References

- [5] <https://docs.python.org/3/using/android.html>
- [6] <https://www.youtube.com/playlist?list=PLSiLeKadTQ7l7DJIUbCj749eAiwIHE4hH>
- [7] <https://forums.raspberrypi.com/viewtopic.php?t=362598>
- [8] <https://buildozer.readthedocs.io/en/latest/installation.html>
- [9] <https://github.com/kivy/python-for-android/blob/develop/doc/source/buildoptions.rst>
- [10] <https://drive.google.com/drive/folders/153rRhW94BhmgXuCopqVUuVe6LwlbACag?usp=sharing>